

CLASE 4 · UNIDAD 1 · SEMANA 4 · PARTE 2

Sentence Embeddings

De la palabra a la oración

El vector de un texto entero, y el motor de recuperación de RAG.

Inteligencia Artificial Generativa Avanzada y Sistemas Multi Agente

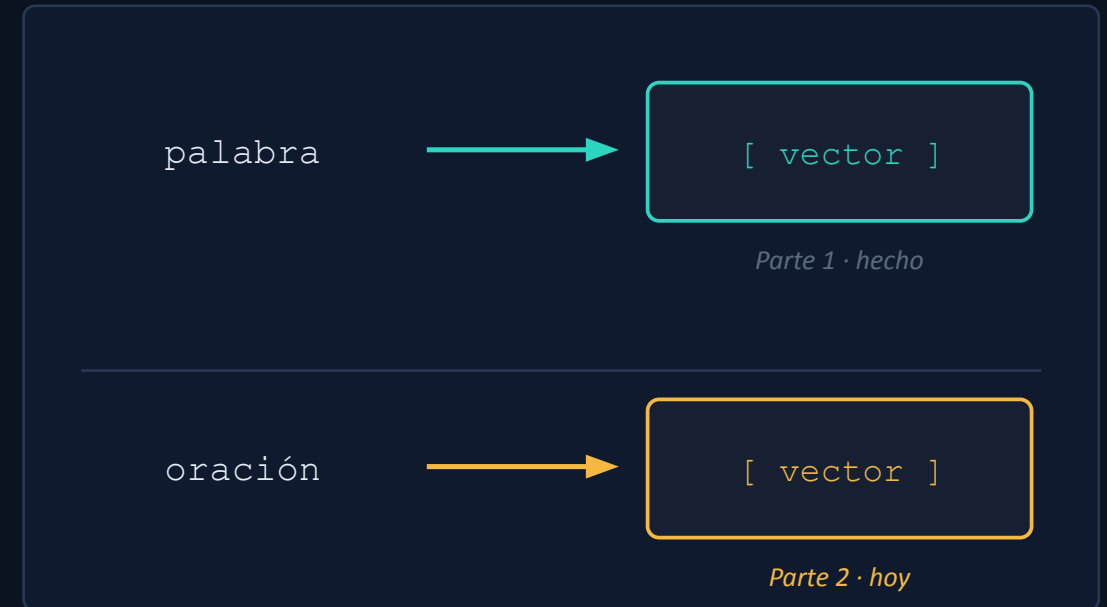
Prof. Francisco Traversaro · Licenciatura en Tecnología Digital, UTDT



Dónde quedamos

En la Parte 1. Conseguimos un vector por palabra: one-hot → contar → word2vec → GloVe. Y cerramos con una bisagra: `most_similar`—los vectores más cercanos por coseno— es, literalmente, la recuperación de RAG.

Lo que falta. Las aplicaciones que nos interesan —una búsqueda, un documento, la pregunta que RAG debe responder— son textos, no palabras sueltas. Para compararlos necesitamos un vector por oración.



El hilo. sentence embeddings + búsqueda semántica = la «R» de RAG (Semana 8). Hoy construimos ese motor.

Hoja de ruta

De la palabra a la oración, y de ahí al motor de RAG. El arco, de a un paso.



La regla que atraviesa todo. La similitud coseno de la Parte 1 (lámina 5) vuelve acá igual: mide dirección entre vectores; sólo que ahora esos vectores representan oraciones, no palabras.

El mapa. Volvemos a esta progresión en el cierre, recorrida entera.

Casi nada es una palabra suelta

El objetivo. Queremos un vector que represente un texto entero —una oración, un párrafo— con un solo vector de largo fijo, igual que GloVe daba un vector de largo fijo por palabra.

Por qué. Las tareas que nos interesan operan sobre textos, no sobre palabras sueltas: comparar dos frases, buscar en una base, usar el texto como feature de otro modelo. Hay que condensar la oración completa en un vector.

“el gato duerme en el sofá”



[un vector de d dim]

¿cómo se arma ese vector?

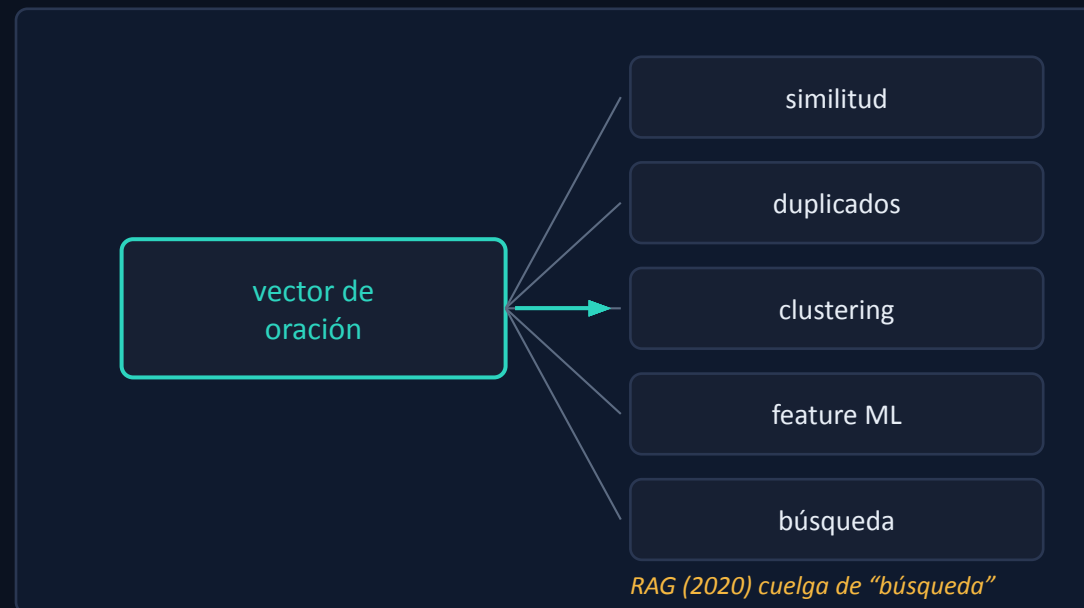
La meta. un vector por oración que capture el significado de todo el texto, no de una palabra.

Para qué sirve un vector de oración

Esto es muy anterior a RAG. Representar un texto con un vector nació para comparar y agrupar textos, mucho antes de los LLM. En cuanto tenés el vector, un montón de tareas se vuelven una cuenta de coseno:

- **Medir similitud** — ¿qué tan parecidas son dos oraciones?
- **Detectar duplicados / paráfrasis** — ¿esta pregunta ya la hicieron?
- **Agrupar por tema** — clustering.
- **Feature engineering** — el vector como feature de un modelo (clasificación o regresión).
- **Buscar por significado** — el texto relevante a una consulta.

La raíz común. apenas necesitás saber “qué tan cerca en significado están dos textos”, necesitás un vector de oración. RAG es un caso de uso, no el origen.



Promediar los word vectors

La idea más simple que puede funcionar. Ya tenemos un vector por palabra (GloVe). El vector de la oración puede ser el promedio de los vectores de sus palabras: sumás los d-dim de cada una y dividís por la cantidad.

Qué da. Un único vector de d dimensiones —el “centro de masa” semántico de la oración— sin entrenar nada nuevo.

$$\text{emb}(\text{oración}) = (v_1 + v_2 + \dots + v_n) / n$$

cinco vectores de palabra entran...



► *notebook §2 — embeber_promedio, coseno entre oraciones.*

Funciona sorprendentemente bien... como piso

El piso. Sin entrenar nada, promediar ya separa lo parecido de lo distinto: “the king rules the country” y “the monarch governs the nation” quedan cerca; “pizza for dinner” queda lejos. Por años fue un baseline difícil de superar.

Pero. tiene tres agujeros que lo hunden en cuanto el texto se pone serio. Los miramos uno por uno — cada uno motiva lo que viene después.

$\cos(\text{king}, \text{monarch}) = \text{alto}$

$\cos(\text{king}, \text{pizza}) = \text{bajo}$

el baseline captura el TEMA. Lo que no captura es todo lo demás.

Agujero 1: ignora el orden

El problema. Sumar es conmutativo: $a + b = b + a$. Dos oraciones con las mismas palabras en distinto orden dan el mismo promedio — aunque signifiquen lo opuesto.

“the dog bit the man” y “the man bit the dog” → mismo vector, $\cos = 1.0$. Quién muerde a quién desaparece.

the dog bit the man

the man bit the dog

$\{the, dog, bit, man\} \rightarrow$ mismas palabras

promedio idéntico → $\cos = 1.0$

La raíz. el orden es significado, y el promedio lo tira a la basura.

► notebook §2 — el caso dog/man, medido.

Agujeros 2 y 3: dilución y OOV

Se diluye con el largo. Una oración larga promedia muchas palabras de relleno (“the”, “of”, “a”) y el significado central se lava. El promedio trata a todas las palabras igual: no sabe cuáles importan.

Descarta lo que no conoce (OOV). Toda palabra fuera del vocabulario de GloVe —un término técnico, un nombre propio— se ignora en silencio. No hay vector, no suma.

DILUCIÓN

“the report is mostly about quantum computing”

vs “quantum computing” → el relleno baja la similitud

OOV

`“pytorch autograd tracks the tensor”`

`pytorch, autograd` no están en GloVe → se ignoran

El diagnóstico. promediar pierde el orden, se diluye y descarta OOV. La raíz: juntamos vectores de palabras aisladas; nunca hubo un modelo que mirara la oración entera.

El Transformer ya sabe leer en contexto

La raíz del problema (lámina 9). promediar junta vectores de palabras aisladas: nadie leyó la oración. Necesitamos algo que la lea en contexto.

Pero eso ya lo construimos. el Transformer lee una secuencia entera con self-attention. En la Clase 2 vimos sus dos mitades:

DECODER (GPT)

genera el próximo token · causal · lo que construimos

ENCODER (BERT)

comprende / representa la secuencia · bidireccional

Clase 2 · apéndice encoder-decoder

El que necesitamos. embeber una oración es comprensión, no generación → vamos al encoder. El encoder-only que nombramos en la Clase 2: BERT.

Qué es BERT, y para qué se usó

Qué es. BERT es ese encoder. Se entrena con masked language modeling: se tapan palabras al azar y el modelo predice las que faltan. Como no genera de izquierda a derecha, puede mirar a los dos lados → bidireccional.

Qué produce. un vector contextual por token: cada palabra, leída según su oración. No genera texto — lo suyo es comprender y representar.

MASKED LANGUAGE MODELING

“el [MASK] duerme en el sofá” → **gato**

CÓMO SE USÓ (2018-2020) · fine-tuning para comprensión

- **clasificación** — sentimiento, spam.
- **NER** — etiquetar personas, lugares.
- **question answering** — marcar la respuesta en un texto.

La clave. BERT es un motor de comprensión — ni generador (eso es GPT), ni medidor de similitud.

Clasificar con BERT: afinado para la tarea

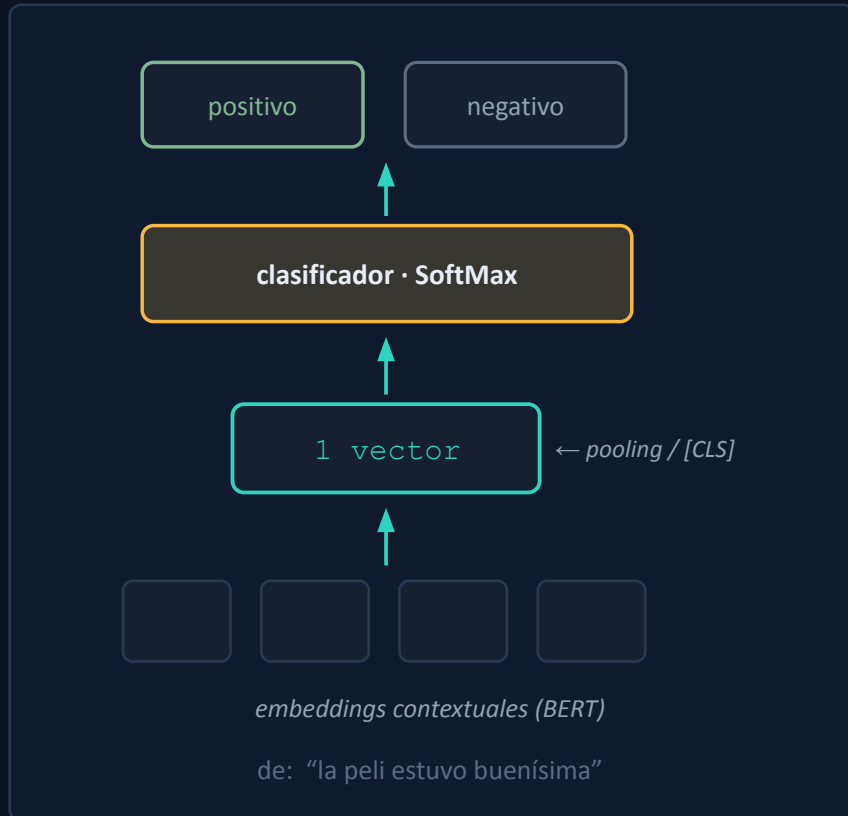


Diagrama recreado · ejemplo basado en StatQuest (BERT).

El detalle que importa. el clasificador (el SoftMax) se entrena junto con BERT para esta tarea: aprende a leer ese vector para sentimiento.

La representación queda atada a la tarea. afinada para “positivo / negativo”, nada más. No es un vector de propósito general.

Lo que nosotros queremos es otra cosa. un embedding de oración de propósito general, cuyo coseno signifique parecido semántico entre dos frases cualesquiera.

El puente. afinar para una tarea es una cosa; un embedding comparable por coseno, otra. Por qué BERT solo no la da → siguiente.

Por qué BERT solo no nos sirve

Nuestro trabajo es otro. no queremos comprender para clasificar; queremos un vector por oración cuyo coseno sea parecido semántico. BERT, tal cual, no da eso.

- **Falla 1.** da un vector por token → hay que poolear a uno por oración.
- **Falla 2 (la grande).** un BERT crudo pooleado es malo para coseno: nunca lo entrenaron para que cerca en significado $\Rightarrow$ cerca en coseno.
- **Falla 3.** ineficiente: para comparar mete las dos oraciones juntas a la red.

EL DATO INCÓMODO

promediar BERT crudo rinde peor que promediar GloVe en STS

EFICIENCIA (del paper)

el par más parecido en 10.000 oraciones $\approx$ 50 M inferencias · ~65 h con BERT

La conclusión. BERT comprende una oración, pero no la representa como un punto comparable por coseno. Hay que adaptarlo → SBERT.

El salto a SBERT: qué le agrega a BERT

El fix, en dos piezas. SBERT (Sentence-BERT) parte del encoder (BERT) y le suma dos cosas. Pooling: de un vector por token a uno por oración. Y —lo central— un entrenamiento contrastivo: ajusta ese vector para que el coseno signifique parecido semántico.

La interfaz. `model.encode("una oración")` → un vector. Todo el trabajo, en una llamada, en la librería `sentence-transformers`

```
from sentence_transformers import \
    SentenceTransformer

m = SentenceTransformer(
    "all-MiniLM-L6-v2")
emb = m.encode(["una oración"])
# emb.shape -> (1, 384)
```

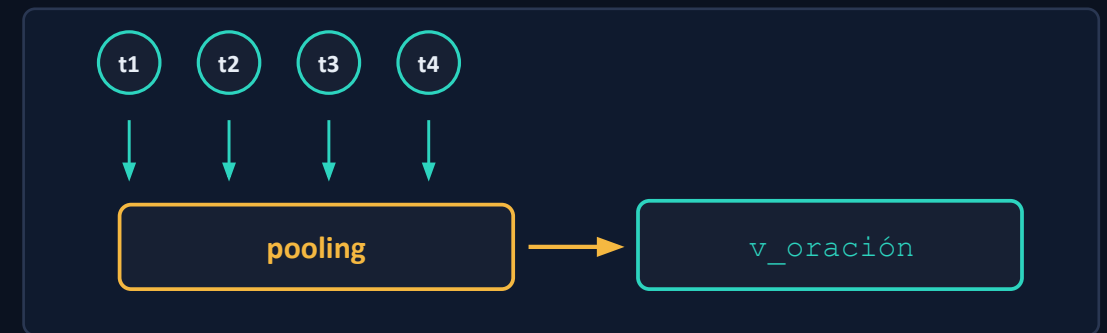
El encuadre. SBERT = BERT + pooling + entrenamiento contrastivo. No es un modelo nuevo: es el encoder que ya conocés, adaptado para dar vectores comparables por coseno.

► *notebook §3 — encode, la forma del vector.*

De tokens a un vector: pooling

El eslabón que faltaba. El encoder da un vector por token, no uno por oración. Falta un paso que colapse esos vectores en uno solo. Ese paso se llama pooling.

Las formas de poolear. promedio de los vectores de token (mean pooling, lo más común), o tomar un token especial: el [CLS] (modelos tipo BERT) o el último token (modelos autoregresivos).



EL TOKEN [CLS]

viene de classification: un token que BERT agrega al inicio de la oración; su vector resume la secuencia entera. Es el que usó para clasificar (lámina 12).

El puente. el encoder da un vector por token; el pooling los junta en uno por oración — el paso que faltaba.

El entrenamiento contrastivo: la idea

El objetivo, otra vez. queremos que $\cos(u, v)$ sea alto si u y v significan lo mismo, y bajo si no. El entrenamiento contrastivo apunta a exactamente eso.

Cómo: siamés (pesos compartidos). el mismo encoder embebe las dos oraciones; una loss compara los dos vectores y ajusta ese único encoder para acercar las parecidas y alejar las distintas.



El aporte de SBERT. BERT crudo no optimizaba la geometría del coseno; SBERT sí — por eso, acá, el coseno significa parecido.

De dónde salen los pares

El dato supervisado. el modelo aprende de pares etiquetados como parecidos (positivos) o no (negativos).

- **Positivos** — paráfrasis, preguntas duplicadas, entailment (NLI), pregunta–respuesta, traducciones.
- **Negativos (el truco)** — cualquier otra oración del batch (in-batch negatives).
Gratis: en un batch de positivos, cada una es negativa de las demás.

Las variantes. in-batch negatives es la receta moderna. Las clásicas de SBERT: NLI (clasificación), STS (regresión), triplet. El detalle fino: fine-tuning.



El payoff. millones de pares → el espacio pasa de disperso a ordenado. Ese orden es lo que hace útil al coseno.

La prueba de fuego: el orden

Volvamos al caso que hundi6 al promedio. “the dog bit the man” vs “the man bit the dog”, ahora con SBERT.

- **promedio de GloVe** → $\cos = 1.0$ (id6nticos, ciego al orden).
- **SBERT** → $\cos < 1$ (registra que el orden, y con 6l el significado, cambi6).

El matiz honesto. el coseno de SBERT no cae a cero: las dos comparten palabras y tema. Pero ya no son id6nticas — que es lo que el promedio no veía.

m6todo	$\cos(\text{dog} \leftrightarrow \text{man})$
promedio	1.0000
SBERT	0.62 aprox.

misma diferencia = el orden capturado

► *notebook §3 — el A/B promedio vs SBERT.*

Promedio vs SBERT

Los tres agujeros del baseline, y cómo los cierra SBERT.

agujero del promedio	cómo lo cierra SBERT
orden (dog/man → mismo vector)	self-attention lee la oración en contexto y en orden
dilución (el relleno lava el sentido)	pondera qué tokens importan; no promedia parejo
OOV (palabras desconocidas se ignoran)	tokeniza en subwords: ninguna palabra queda “afuera”

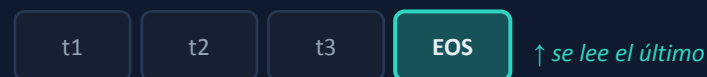
Qué compramos. pasar de “bolsa de vectores promediada” a “un modelo que lee la oración entera” — y un coseno que, ahora sí, significa parecido semántico.

El mismo truco en un GPT (decoder)

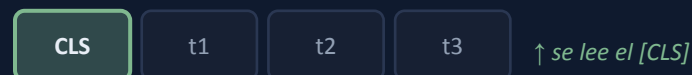
El mismo camino, otro backbone. SBERT parte de un encoder (BERT); la misma receta —vectores contextuales → poolear a uno → afinar contrastivo— corre igual sobre un decoder (un GPT, el que construimos). El GPT crudo no da un buen embedding: el entrenamiento lo vuelve bueno.

La única diferencia: la máscara. el encoder es bidireccional → el vector se lee del [CLS]. El decoder es causal (cada token ve sólo el pasado) → el único que vio toda la oración es el último; ese se lee.

GPT · causal (mira sólo el pasado)



BERT · bidireccional (mira todo)



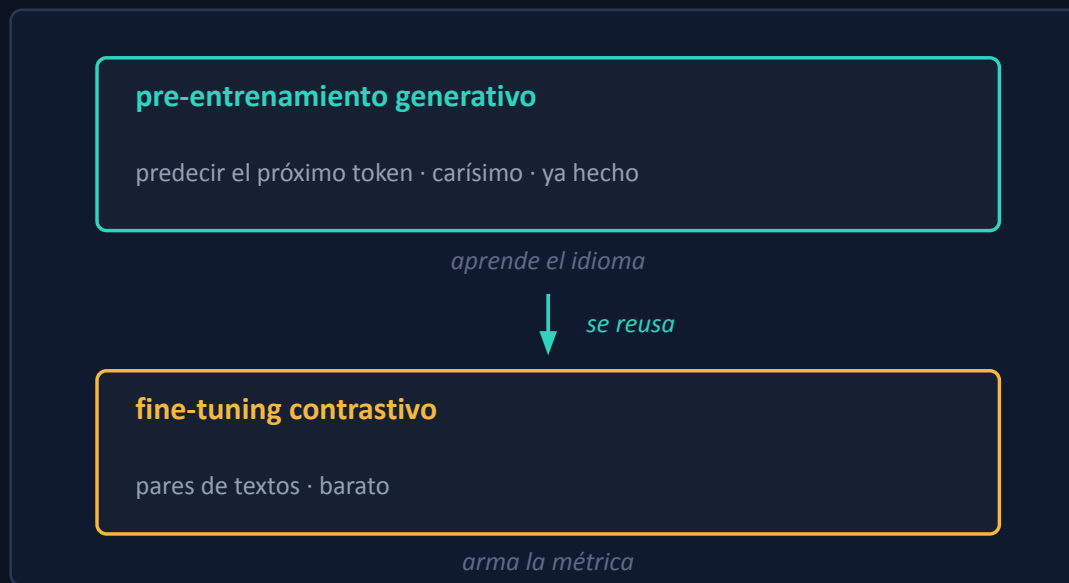
ambos → pooling + fine-tuning contrastivo → embedding

`OpenAIEmbeddings` por dentro. init desde un GPT, se lee el último token, objetivo contrastivo. Un modelo dedicado, distinto del GPT de chat — vale para cualquier embedder basado en decoder.

De un GPT a un embedder: no se empieza de cero

El punto de partida. el embedder no se entrena desde ruido: arranca del GPT ya pre-entrenado —el mismo pre-entrenamiento generativo de predecir el próximo token— y sobre eso corre el fine-tuning contrastivo.

Por qué reusarlo. para predecir bien el próximo token, la red tuvo que aprender sintaxis, semántica, qué se relaciona con qué, mundo. Ese conocimiento ya vive en los pesos. Pre-entrenar es el paso carísimo (miles de GPUs, billones de tokens); el contrastivo, al lado, es barato.

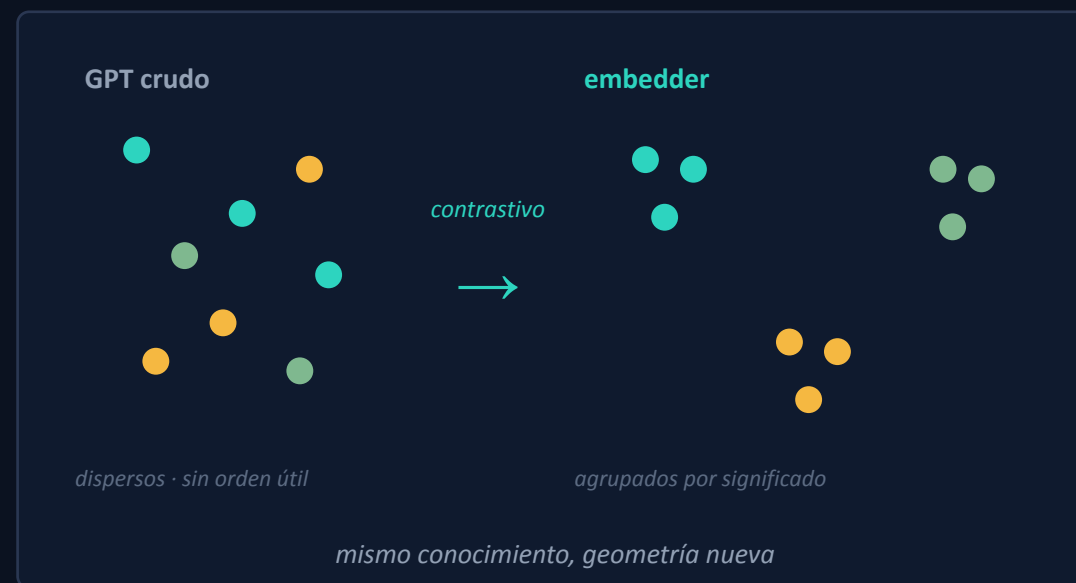


No es empezar de cero — ni congelar. el contrastivo sí actualiza los pesos del backbone, pero arrancando de los valores pre-entrenados, no de ruido. Un ajuste —pocos pares, pocas épocas—, por eso es barato.

Conocimiento vs métrica: por qué el vector crudo no alcanza

El problema del vector crudo. el GPT pre-entrenado ya tiene representaciones riquísimas — pero el hidden state del último token está optimizado para predecir el próximo token, no para que el coseno mida parecido. Crudo, la geometría no sirve para buscar: dos textos con el mismo significado pueden quedar lejos.

Qué hace el contrastivo. no le enseña el idioma de nuevo: reordena el espacio. Reexpresa el conocimiento que ya está en los pesos en una geometría donde coseno = significado. Es el salto BERT → SBERT, sobre un decoder.



El pre-entrenamiento pone el conocimiento; el contrastivo pone la métrica. dos textos que significan lo mismo quedan cerca sólo después del paso contrastivo.

Cuidado con la palabra “embedding”

Tres cosas distintas se llaman “embedding”. no las confundas:

- **De entrada** — un vector fijo por token del vocabulario; un lookup. Es la capa de entrada del modelo.
- **Contextual por token** — la salida de las capas con atención: un vector por token que ya leyó su contexto. Sigue habiendo uno por token.
- **Del texto (el de RAG)** — un solo vector para la query o el documento: el contextual del último token (GPT) o del [CLS] (BERT), ya afinado.

LOS TRES “EMBEDDINGS”

entrada · vector fijo por token (lookup)

uno por token

contextual · por token, ya con contexto

uno por token

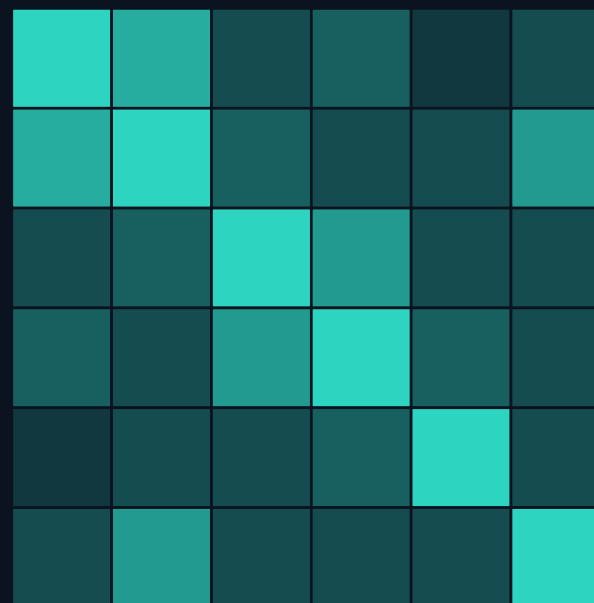
del texto (RAG) · UNO por texto

El que se usa para buscar es el tercero. los otros dos son piezas internas. “El embedding de la query” es el del texto — uno solo, y recién útil tras el entrenamiento contrastivo.

Coseno entre oraciones: el mapa de conceptos

El corpus, y el coseno. armamos una base de conocimiento —una definición corta por concepto del curso (token, embedding, softmax, RAG...), lo que un RAG guarda— y calculamos el coseno entre todas. Los conceptos que hablan de lo mismo se iluminan juntos: word2vec y negative sampling, o embedding y coseno.

Lo importante. nadie le dijo al modelo qué conceptos se relacionan. Lo dedujo del significado de las definiciones. Esa geometría —significados parecidos, vectores cercanos— habilita el paso siguiente.



similitud coseno entre conceptos

► *notebook §4 — el corpus, la matriz de coseno y el heatmap.*

Búsqueda semántica = la «R» de RAG

El payoff. llega una query, la embebemos con el mismo modelo que el corpus, y devolvemos los documentos de mayor coseno. Query + documentos → coseno → top-k.

Eso es la recuperación de RAG. tal cual. Es lo que la Semana 8 construye en serio, sobre bases grandes.



► *notebook §5 — buscar a mano y util.semantic_search.*

Semántico, no por palabras

La diferencia que importa. una query puede no compartir casi ninguna palabra con el documento correcto y, aun así, recuperarlo primero. El match es por significado, no por vocabulario.

El cabo suelto de la Clase 1. el retriever por palabras clave de la Clase 1 fallaba con sinónimos; la búsqueda semántica lo resuelve.

► *notebook §5 — la query sin palabras en común.*

EN EL CORPUS DE CONCEPTOS

query

“cómo le doy a un modelo info que no está en su entrenamiento”

↓ *mejor coseno del corpus*

top-1 · definición de “RAG”

“traer texto de una base externa y pasárselo al modelo para responder”

palabras en común ≈ 0 · mismo significado

Sin RAG vs con RAG: qué aporta

Cómo se ve qué aporta. si preguntás algo que el modelo ya sabe, no notás la diferencia. RAG se luce cuando el dato no está en sus pesos. Preguntamos por un hecho de la materia que el modelo no puede conocer:

- **sin RAG** → el modelo inventa una fecha o esquiva.
- **con RAG** → recupera el dato de la base y responde con él.

El loop de la Clase 1, cerrado. el retriever naive por keywords ahora es semántico, y encima genera sobre lo recuperado.

► *notebook §6 — el contraste sin/con RAG.*

“¿cuándo se entrega el TP1?”

sin RAG

“...no tengo esa información” / inventa

con RAG

“el 27 de septiembre” (del doc recuperado)

La invariante: cada modelo tiene su propio espacio

La regla. la query y los documentos tienen que embeberse con el mismo modelo. El coseno sólo tiene sentido dentro de un mismo espacio vectorial.

Por qué. cada embedder se entrena por su cuenta y arma su propio espacio, sin relación con el de otro. Corpus con un modelo y query con otro → universos distintos → ranking basura. A veces ni la dimensión coincide (384 vs 768). Cambiar de embedder obliga a re-indexar todo.

modelo A · 384

modelo B · 768

$$\cos(q_A, q_B) \approx 0$$

espacios incompatibles

En una línea. un embedder, un espacio — el mismo para la base y para la query, o el coseno no significa nada.

► *notebook §5 — la misma query con dos modelos.*

Simétrico vs asimétrico: query vs documento

El mismo vector, dos trabajos. la librería suele ser la misma; lo que cambia es cómo son la consulta y el objetivo.

- **Simétrico — feature engineering (ML)** — cada texto para clasificar o agrupar. Consulta y texto, mismo tipo y largo.
- **Asimétrico — RAG / retrieval** — una query corta busca un pasaje largo que la responde. Se entrena con pares query–pasaje; conviene un modelo de retrieval.

simétrico

“¿estas dos frases son parecidas?”

asimétrico

query corta → ¿qué pasaje la responde?

algunos piden un prefijo: `query:` / `passage:`

La constante. en los dos casos, un solo modelo para todo el pipeline (la invariante de la lámina 28).

Qué embedder usar para RAG (2026)

La guía práctica — acá sí, nombres.

- **El benchmark:** MTEB — el ranking estándar para comparar embedders.
- **Pagos por API** (OpenAI, Cohere, Google) — fuertes, pero atados a un proveedor y con costo por token.
- **Open que ya compite:** Qwen3-Embedding, BGE-M3 (multilingüe) alcanzan a los comerciales.
- **Livianos y locales:** nomic-embed-text (137M, corre en CPU), bge-m3 (multilingüe, contexto largo).

cómo elegir

- **idioma** ¿multilingüe?
- **tamaño / latencia** ¿entra en tu hardware?
- **licencia** ¿uso comercial?

La regla al elegir. benchmarkeá en tus datos: el ranking general no siempre predice tu caso.

Los tres modelos de un RAG

Tres roles, independientes. un agente con RAG tiene: un LLM generador (escribe la respuesta), un embedder que indexó la base, y un embedder que embebe la query.

La única atadura dura: índice = query. el de la query y el de los documentos, el mismo —mismo espacio, o el coseno no significa nada (la invariante de la lámina 28). Cambiar de embedder → re-indexar toda la base.

El generador, aparte. nunca ve vectores: recibe el texto de lo recuperado. Cambiás el LLM sin re-embeber nada.

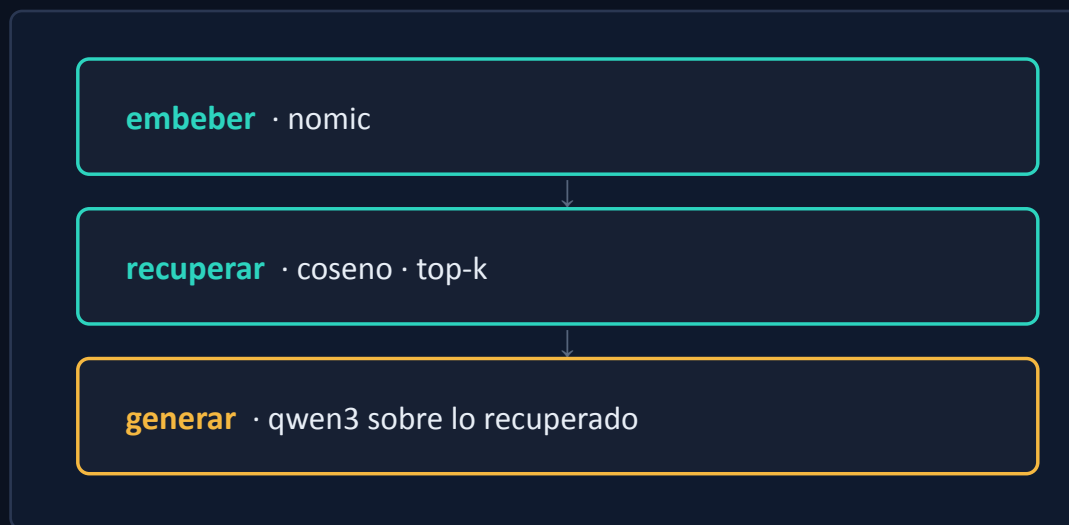


Un espacio para buscar, un modelo para escribir. se comunican por texto, no por vectores.

De retrieval a RAG: el stack mínimo

Lo que falta. retrieval devuelve documentos; RAG los usa para generar una respuesta. Juntamos las piezas, todas locales y libres.

Eso es RAG. recuperar + generar sobre lo recuperado. El reemplazo directo del stack con `OpenAIEmbeddings`, sin API key ni proveedor atado.



► *notebook §6 — responder(pregunta, usar_rag).*

Lo que falta para RAG en serio

Esto es el esqueleto. funciona, pero es mínimo. Lo que lo hace escalar —y es la Unidad 4, Semana 8:

- **Chunking** — partir documentos largos en pedazos indexables.
- **Vector store** de verdad (Chroma, pgvector, Qdrant) en vez de una lista de numpy.
- **Reranking** — un segundo paso que afina el top-k.

Lo que ya tenés. la pieza central —embeber y recuperar por coseno— es exactamente lo de hoy.

El puente. de este esqueleto a un RAG de producción, lo que falta es escala, no concepto.

HOY

embeber + recuperar (coseno)

SEMANA 8 le suma

chunking

vector store

reranking

Síntesis: el mapa de la Parte 2

El mapa del principio, ahora recorrido entero.

paso	idea	herramienta
palabra → oración	necesitamos un vector por texto	—
promediar	baseline; y sus agujeros (orden, dilución, OOV)	GloVe (P1)
SBERT	un modelo que lee la oración: pooling + entrenamiento	sentence-transformers
coseno entre oraciones	los significados parecidos caen cerca	la regla de P1
búsqueda semántica	query + docs → coseno → top-k = la «R» de RAG	util.semantic_search
RAG mínimo	recuperar + generar, todo local y libre	Ollama (nomic + qwen3)

La regla transversal. un embedder = un espacio. Query y corpus, siempre con el mismo.

La bisagra, cerrada

Lo que preguntaba la Parte 1. de dónde salen los vectores que representan texto.

Lo que respondió la Parte 2. los llevamos de la palabra a la oración, vimos que un modelo (no un promedio) captura significado, y montamos búsqueda semántica sobre eso — el motor de recuperación de RAG.

De acá a RAG completo: sólo falta escala (chunking, vector store, reranking). El concepto ya está.



El arco. one-hot (identidad) → static (significado) → contextual (Transformer) → oración (SBERT) → recuperación (la «R» de RAG).

Trabajo y lecturas

NOTEBOOK — N2 · SENTENCE EMBEDDINGS

El arco de hoy, ejecutable de punta a punta.

- 1 De la palabra a la oración: el problema (§1)
- 2 Baseline: promediar, y sus agujeros (§2)
- 3 SBERT: encode, el A/B del orden, multilingüe (§3)
- 4 Coseno entre oraciones: corpus + heatmap (§4)
- 5 Búsqueda semántica + la invariante (§5)
- 6 RAG mínimo Ollama-nativo: sin/con RAG (§6)

LECTURAS

Núcleo

- Reimers & Gurevych, 2019 — Sentence-BERT (la red siamesa)
- Documentación de sentence-transformers — semantic search (simétrico vs asimétrico)
- Documentación de embeddings de Ollama

Para afianzar

- Neelakantan et al., 2022 — Text and Code Embeddings by Contrastive Pre-Training
- El leaderboard MTEB — comparar embedders

Lo que sigue. RAG desde cero — chunking, vector store, reranking (Unidad 4, Semana 8).